

API en PaaS

Semana: 8

Nombre del estudiante:

Tania Nayeli Oliva Murillo_62321160

Heber Enoc Peña Pineda_61941442

Milton Leonel Ortiz_61841317

Sede de estudio:

Sede Central

Docente:

María José Salinas Quiroz

Sección:

574

Fecha de entrega:

13 de septiembre de 2026

Índice

Documento de decisión: arquitectura y plataforma PaaS para ClinData.....	3
Introducción	3
Contexto y criterios de comparación	3
Plataforma elegida: Azure App Service.....	4
Arquitectura de despliegue: monolito.....	5
Organización interna: N-Capas	5
Entidades y reglas de negocio	6
Endpoints de la primera versión	6
Conclusión	7
Referencias	7

Documento de decisión: arquitectura y plataforma PaaS para ClinData

Introducción

ClinData es una API REST orientada a la gestión de expedientes clínicos para un médico independiente. El sistema permitirá registrar pacientes, agendar citas y almacenar notas clínicas, priorizando la integridad, trazabilidad y seguridad de la información. En este documento se analizan las necesidades del caso para seleccionar una arquitectura de software y una plataforma PaaS adecuadas. También se presentan las entidades, reglas de negocio y endpoints previstos para la primera versión de la aplicación.

Contexto y criterios de comparación

ClinData es el expediente digital de un médico que trabaja por su cuenta. Del enunciado salen tres condiciones que mandan sobre todo lo demás: una nota clínica escrita ya no se edita ni se borra, el consultorio atiende un volumen bajo de pacientes, y el sistema tiene que poder mantenerse y auditarse por años. Con eso definimos los criterios con los que comparamos las plataformas.

- Que la API siga respondiendo sin que nadie la levante, porque la entrega pide una URL funcional el día de la revisión.
- Que corra .NET sin que tengamos que armar el runtime, ya que el equipo cuenta con 27 horas en total.
- Que la base de datos administrada no caduque. Un expediente que promete trazabilidad no puede guardarse en un almacenamiento con fecha de vencimiento.

Tabla 1

Comparación de Azure App Service y Render según los criterios del caso

Criterio	Azure App Service (F1)	Render (plan gratuito)
Continuidad del servicio	Gratuito de forma permanente, con 60 minutos de CPU al día y 1 GB de memoria. No trae Always On, pero no apaga la aplicación por una ventana fija de inactividad.	El servicio web se suspende a los 15 minutos sin tráfico y la siguiente petición tarda cerca de un minuto en responder.
Soporte de .NET	El runtime se elige al crear la aplicación y el despliegue sale desde Visual Studio, GitHub Actions o un paquete comprimido.	No tiene runtime nativo de .NET. Hay que empaquetar la API en un contenedor y mantener el Dockerfile.
Base de datos administrada	Azure SQL Database serverless entra en la capa siempre gratuita, con 100,000 segundos de vCore al mes, y no caduca.	La base PostgreSQL gratuita expira 30 días después de creada y luego se elimina con sus datos.

Nota. Datos tomados de la documentación oficial de Microsoft y de Render, consultada en septiembre de 2026.

Plataforma elegida: Azure App Service

El tercer criterio fue el que decidió. Render borra la base de datos gratuita a los 30 días, y no tiene sentido prometer trazabilidad clínica sobre algo que se elimina solo. El segundo criterio apunta en la misma dirección: en Render, .NET únicamente entra por contenedor, así que gastaríamos horas escribiendo y arreglando un Dockerfile que no suma puntos en la rúbrica.

El apagado por inactividad también pesó. Si la docente corre la colección de Postman después de un rato sin tráfico, en Render la primera petición demora alrededor de un minuto y los pm.test que validan el código de estado fallarían por tiempo de espera. La contraparte de F1 hay que decirla: 60 minutos de CPU diarios y sin dominio propio. Para el tráfico de este proyecto la cuota sobra, y la URL de azurewebsites.net cumple con lo que pide la entrega. Si en la semana del despliegue el consumo apretara, el crédito de Azure for Students permite subir el plan a B1 sin tocar el código.

Las otras opciones quedaron fuera antes de la tabla: Heroku y Railway ya no tienen capa gratuita, y el entorno estándar de Google App Engine no ejecuta .NET.

Arquitectura de despliegue: monolito

No encontramos ninguna razón para partir ClinData en microservicios. No hay áreas que cambien a ritmos distintos, no hay equipos separados desplegando por su cuenta y el propio caso descarta el tráfico alto. Hay además un argumento en contra de separarlo: las reglas que más le importan al médico son de consistencia entre entidades, como que una nota solo pueda existir si su cita existe y ya ocurrió. Eso se resuelve con una transacción sobre una sola base de datos. Repartido en servicios, cada validación se convierte en una llamada de red y la consistencia pasa a ser eventual, que es justo lo que el caso no tolera.

Organización interna: N-Capas

Adentro del monolito el código va en capas y no en MVC. La API devuelve datos, no vistas, así que el patrón se queda sin la V y el controlador termina cargando lógica que no le corresponde. El médico también pide poder auditar el sistema, y auditar significa responder con precisión dónde se garantiza que una nota clínica no se modifica. Si esa garantía está repartida entre el controlador, el contexto de datos y una validación de modelo, cualquier cambio futuro puede romperla sin que nadie se dé cuenta. Es el mismo razonamiento que aplicamos al migrar BibliotecaMonolito hacia Clean Architecture, cuando la detección de autores duplicados salió del controlador y quedó en su propio validador. La solución queda en cuatro proyectos:

- **ClinData.API:** controladores, rutas y códigos de respuesta. Sin reglas de negocio.
- **ClinData.Application:** servicios y validadores. Aquí vive el checklist del caso.
- **ClinData.Domain:** entidades e invariantes propias, sin dependencias externas.
- **ClinData.Infrastructure:** DbContext de EF Core, repositorios y conexión a Azure SQL Database.

Entidades y reglas de negocio

El modelo se reduce a tres entidades con relación lineal: un paciente tiene varias citas y cada cita puede tener una sola nota clínica. El historial del paciente se consulta mediante la relación NotaClinica Cita Paciente, sin duplicar PacienteId en NotaClinica.

Tabla 2

Entidades, atributos y validadores responsables de cada regla

Entidad	Atributos	Reglas y dónde se validan
Paciente	Id, Identidad, Nombres, Apellidos, FechaNacimiento, Telefono, Sexo, FechaRegistro	ValidadorPaciente revisa el formato 0801-1990-12345, más un índice único en la base de datos para que no se repita.
Cita	Id, PacienteId, FechaHora, Motivo, Estado (Programada, Atendida, Cancelada)	ValidadorCita rechaza fechas y horas pasadas, y evita que un paciente tenga dos citas en el mismo horario.
NotaClinica	Id, CitaId, Texto, FechaHoraRegistro, RegistradaPor.	ValidadorNotaClinica exige que la cita exista, que ya haya ocurrido y que RegistradaPor venga lleno. La entidad no expone modificadores públicos y la API no publica rutas para editarla ni eliminarla. Además, CitaId tendrá un índice único para garantizar que cada cita tenga una sola nota clínica.

Endpoints de la primera versión

Tabla 3

Endpoints previstos para la API de ClinData

Método	Ruta	Función
POST	/api/pacientes	Registrar un paciente.
GET	/api/pacientes	Listar pacientes.
GET	/api/pacientes/{id}	Consultar un paciente.
GET	/api/pacientes/{id}/notas	Listar el historial de notas del paciente.
POST	/api/citas	Agendar una cita.
GET	/api/citas?fecha=	Listar las citas de una fecha.
GET	/api/citas/{id}	Consultar una cita.
POST	/api/notas	Registrar una nota clínica asociada a una cita.
GET	/api/notas/{id}	Consultar una nota clínica.

Nota. No hay rutas PUT ni DELETE sobre notas clínicas. Su ausencia es una decisión de diseño que viene de la regla de inmutabilidad, no un pendiente de implementación.

Conclusión

Después de analizar las necesidades de ClinData, se determinó que un monolito organizado en N-Capas es la opción más adecuada, debido al volumen moderado de usuarios y a la necesidad de mantener las responsabilidades claramente separadas. Azure App Service, junto con Azure SQL Database, fue seleccionado para el despliegue por su compatibilidad con .NET y la permanencia de su oferta gratuita de base de datos. La solución propuesta permitirá desarrollar una API comprensible, mantenible y auditable, garantizando especialmente que las notas clínicas no puedan modificarse ni eliminarse después de su registro.

Referencias

Microsoft. (2026). Documentación de Azure App Service. Microsoft Learn.

<https://learn.microsoft.com/es-es/azure/app-service/>

Microsoft. (2026). Límites, cuotas y restricciones de suscripción y servicios de Azure. Microsoft

Learn. <https://learn.microsoft.com/es-es/azure/azure-resource-manager/management/azure-subscription-service-limits>

Microsoft. (2026). Precios de App Service. Microsoft Azure. [https://azure.microsoft.com/es-](https://azure.microsoft.com/es-es/pricing/details/app-service/windows/)

[es/pricing/details/app-service/windows/](https://azure.microsoft.com/es-es/pricing/details/app-service/windows/)

Microsoft. (2026). Arquitectura de microservicios .NET para aplicaciones .NET en

contenedores. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/architecture/microservices/>

Render. (2026). Free instance types. Render Docs. <https://render.com/docs/free>